DREAM CATCHER

Intent-Based 5G Network Slice Controller

VERSION 1.0.0

User Manual

DreamCatcher

Project Title	Dream Catcher
Version	1.0.0
Team Members	Wayne Tsai, Marjorie Yang, Yourong Xu
Instructor	Luyao Niu, Mark Licata
Industrial Sponsor	T-Mobile
Sponsor Contact	Justin Ho
Submission Date	March 12, 2026

© 2026 Dream Catcher Team — Sponsored by **T-Mobile**

CONFIDENTIAL

Contents

1 Product Overview 2
- 1.1 What Is Dream Catcher? 2
- 1.2 The Problem It Solves 2
- 1.3 Who Is It For? 2
- 1.4 Primary Features 2

2 System Requirements 3
- 2.1 Supported Operating Systems 3
- 2.2 Hardware Requirements 3
- 2.3 Software Dependencies 3
- 2.4 Internet Requirements 4
- 2.5 Network Ports Used 4

3 Installation and Setup 4
- 3.1 Get the Code 4
- 3.2 Install Node.js 4
- 3.3 Install Each Component 4
- 3.4 Install Python Dependencies (Live Mode Only) 5
- 3.5 Configure pfSense Credentials (Live Mode Only) 5
- 3.6 pfSense API Package Setup 5

4 Operating Instructions 6
- 4.1 Choosing a Run Mode 6
- 4.2 Mode A — TUI Demo Only 6
- 4.3 Mode B — Full Stack Demo (Recommended for Presentations) 7
- 4.4 Mode C — Backend with Mock Data and Dashboard 8
- 4.5 Mode D — End-User Dashboard 8
- 4.6 Mode E — Live Mode (Real Traffic) 9
- 4.7 Switching Dashboard Views 10
- 4.8 Shutting Down 10

5 Technical Specifications 10
- 5.1 System Architecture 10
- 5.2 Technologies Used 11
- 5.3 Data Flow 11
- 5.4 WebSocket Message Format 11
- 5.5 Intent Classification Categories 12
- 5.6 Demo Scenario Timeline 12

6 Security and Privacy 12
- 6.1 API Key Security 12
- 6.2 Network Access 13
- 6.3 TLS and the pfSense Certificate 13
- 6.4 Packet Capture 13
- 6.5 Data Privacy 13
- 6.6 Known Operational Security Limitations 13

7 Troubleshooting 13
- 7.1 The Backend Fails to Start 14
- 7.2 The Frontend Shows No Data 14
- 7.3 The TUI Shows a Blank Screen or Crashes 14
- 7.4 Live Mode Fails with "Permission Denied" 14
- 7.5 pfSense API Returns 401 Unauthorized 14
- 7.6 pfSense API Returns 403 Forbidden 15
- 7.7 Gateway Group Name Not Found 15

7.8 TUI Not Forwarding Events to the Backend 15
7.9 NFStream Fails to Install 15

8 Limitations **16**

8.1 Demo Mode vs. Live Mode 16
8.2 macOS Support for Live Mode 16
8.3 No Authentication Layer 16
8.4 Single Gateway Group 16
8.5 In-Memory State Only 16
8.6 Fixed Intent Mapping 16
8.7 Throughput Measurement Accuracy 16
8.8 Browser Compatibility 16
8.9 pfSense Version Compatibility 16

1 Product Overview

1.1 What Is Dream Catcher?

Dream Catcher is an intelligent network management system for 5G home and small-office routers. It watches the network traffic on your connection in real time, determines what kind of activity is happening — a video call, a gaming session, or regular browsing — and automatically switches the router to the best possible network configuration for that activity, without any manual input from the user.

Think of it as a smart traffic controller living inside your router. When it detects that you have joined a Zoom meeting, it instantly prioritises the low-latency network path for that call. When the meeting ends, it quietly switches back to normal settings.

1.2 The Problem It Solves

Standard home routers treat all network traffic identically. A video call competes equally with a background file download or a software update. This causes dropped frames, choppy audio, and degraded call quality at the moments that matter most.

Dream Catcher eliminates this problem by continuously monitoring traffic, identifying high-priority applications, and dynamically reassigning network resources — automatically and in real time.

1.3 Who Is It For?

Audience	Use Case
Home users	Automatic quality improvement for video calls and gaming without any configuration
IT administrators	Monitor and control network slice behaviour across a pfSense-managed network
Business stakeholders	Visualise the measurable quality-of-experience improvement driven by intelligent slice switching
Network engineers	Extend or customise the intent classification and WAN switching logic

1.4 Primary Features

- **Automatic traffic classification** — identifies applications (Zoom, Teams, FaceTime, YouTube, Steam, etc.) by their network behaviour using deep packet inspection
- **Real-time WAN switching** — switches the active WAN gateway on a pfSense router the moment high-priority traffic is detected
- **Terminal UI** — a fullscreen live view of all network flows, current slice state, and network metrics
- **End-user dashboard** — a browser-based interface for home users to monitor router status and configure priority rules
- **Business dashboard** — a stakeholder-facing view showing quality-of-experience charts with before/after switch markers and business impact metrics
- **Demo mode** — a fully self-contained simulation requiring no router hardware, suitable for

presentations and development

2 System Requirements

2.1 Supported Operating Systems

Component	Supported OS
Backend	macOS 12+, Linux (Ubuntu 20.04+, Debian 11+)
Frontend dashboards	Any OS with a modern browser (Chrome 110+, Firefox 110+, Safari 16+)
TUI — demo mode	macOS 12+, Linux
TUI — live mode	Linux only (raw packet capture; macOS has NFStream limitations)

2.2 Hardware Requirements

Minimum (demo mode / dashboards only):

- CPU: dual-core 2 GHz
- RAM: 4 GB
- Disk: 500 MB free

Live mode (full system):

- CPU: quad-core 2 GHz recommended (NFStream is CPU-intensive during capture)
- RAM: 8 GB
- Network: a second network interface or SPAN/mirror port for packet capture
- Router: pfSense 2.7+ with two configured WAN gateways and a gateway group

2.3 Software Dependencies

All components:

- Node.js 18 or later
- npm 9 or later

TUI live mode only:

- Python 3.12 or later
- pip packages: `nfstream`, `pyyaml`, `requests`, `scapy`, `psutil`
- Root/sudo privileges for raw packet capture

pfSense router (live mode only):

- pfSense 2.7+

- pfSense-pkg-API package installed
- Two WAN gateways configured under **System → Routing → Gateways**
- A gateway group configured under **System → Routing → Gateway Groups**
- REST API enabled and an API key generated under **System → API → Keys**

2.4 Internet Requirements

- **Demo mode:** no internet connection required
- **Live mode metrics:** uses ICMP ping to public DNS servers (8.8.8.8, 1.1.1.1) — requires outbound internet access
- **Active throughput measurement:** downloads from the Cloudflare speed test endpoint

2.5 Network Ports Used

Service	Port	Protocol
Backend REST API	3002	HTTP
Backend WebSocket	3002	WS
Business / User frontend	5173	HTTP
User frontend local API	3001	HTTP

3 Installation and Setup

3.1 Get the Code

```
git clone <repository-url>
cd final-codebase-dreamcatcher
```

3.2 Install Node.js

If Node.js is not already installed:

- **macOS:** `brew install node` (requires Homebrew)
- **Linux:** use nvm or your distribution's package manager

Verify the installation:

```
node --version    # should print v18.x.x or higher
npm --version
```

3.3 Install Each Component

Each component is a standalone Node.js project installed independently:

```
# Backend
cd dream-catcher-backend && npm install

# Business Dashboard
cd dream-catcher-frontend-business && npm install

# End-User Dashboard
cd dream-catcher-frontend-user && npm install

# TUI
cd dream-catcher-tui && npm install
```

3.4 Install Python Dependencies (Live Mode Only)

```
pip install nfstream pyyaml requests scapy psutil
```

Note: On some systems you may need `pip3` instead of `pip`. NFStream and Scapy both require Python 3.12 or later.

3.5 Configure pfSense Credentials (Live Mode Only)

Before running the TUI in live mode, set your pfSense connection details in `dream-catcher-tui/tools/wan_toggle-v2.py`:

```
# --- Edit these values ---
API_BASE_URL = "https://<your-pfsense-ip>/api/v2"
API_KEY      = "YOUR_PFSENSE_API_KEY"

GATEWAY_GROUP_NAME = "WAN_FAILOVER"   # must match your pfSense gateway group
    name
WAN1_GATEWAY_NAME  = "WAN1_DHCP"      # must match your WAN1 gateway name
WAN2_GATEWAY_NAME  = "WAN2_DHCP"      # must match your WAN2 gateway name
```

Gateway names can be found in pfSense under **System → Routing → Gateways**.

Important: Never commit your API key to source control. The placeholder `YOUR_PFSENSE_API_KEY` must be replaced locally only and must never be pushed to any shared repository.

3.6 pfSense API Package Setup

1. Log in to your pfSense web interface.
2. Navigate to **System → Package Manager → Available Packages**.
3. Search for **pfSense-pkg-API** and click **Install**.
4. After installation completes, go to **System → API → Keys**.
5. Click **Add** to generate a new API key. Copy the key — paste it into `wan_toggle-v2.py` as shown above.

6. Confirm the API is enabled and that HTTPS access is permitted from the machine running the TUI.

4 Operating Instructions

4.1 Choosing a Run Mode

Dream Catcher supports several operating configurations. Choose the one that best fits your goal:

Mode	What Runs	Best For
A — Demo: TUI only	TUI in demo mode	Seeing the terminal UI without any hardware
B — Demo: Full stack	TUI + Backend + Business Dashboard	Full end-to-end presentation demo
C — Mock: Backend + Dashboard	Backend (mock) + Business Dashboard	Frontend development, demo without TUI
D — End-user dashboard	User frontend + its local server	End-user monitoring view
E — Live: Full system	TUI (live) + Backend + Dashboards	Production use with a real pfSense router

4.2 Mode A — TUI Demo Only

The simplest way to see Dream Catcher in action. No backend or browser required.

Step 1. Open a terminal and navigate to the TUI directory:

```
cd dream-catcher-tui
```

Step 2. Start the demo:

```
npm run dev -- --demo
```

What you will see: The terminal switches to fullscreen mode showing four panels:

```
+-- Network Traffic ----------------------------++-- Slice Status ------+
| (scrolling list of detected flows, color-     || Active: WAN1 (Normal)|
|  coded by intent category)                    || Phase:  IDLE         |
|                                               |+-- Network Metrics ---+
|                                               || Latency   18.2ms     |
|                                               || Jitter     2.1ms     |
|                                               |+-- Slice Events ------+
|                                               || (timestamped log)    |
+-----------------------------------------------+----------------------+
```

The demo runs a 45-second cycle automatically. After roughly 10 seconds, a Zoom flow appears, the slice switches to WAN2 (low-latency), metrics improve, and all panels update in real time. Press **q** to quit.

4.3 Mode B — Full Stack Demo (Recommended for Presentations)

Wires the TUI → Backend → Business Dashboard together for a complete end-to-end demo. Open three terminal windows.

Terminal 1 — Start the backend in TUI ingest mode:

```
cd dream-catcher-backend
TELEMETRY_SOURCE=tui npm run dev
```

Wait until you see: `Server running on http://127.0.0.1:3002`

Terminal 2 — Start the TUI, forwarding events to the backend:

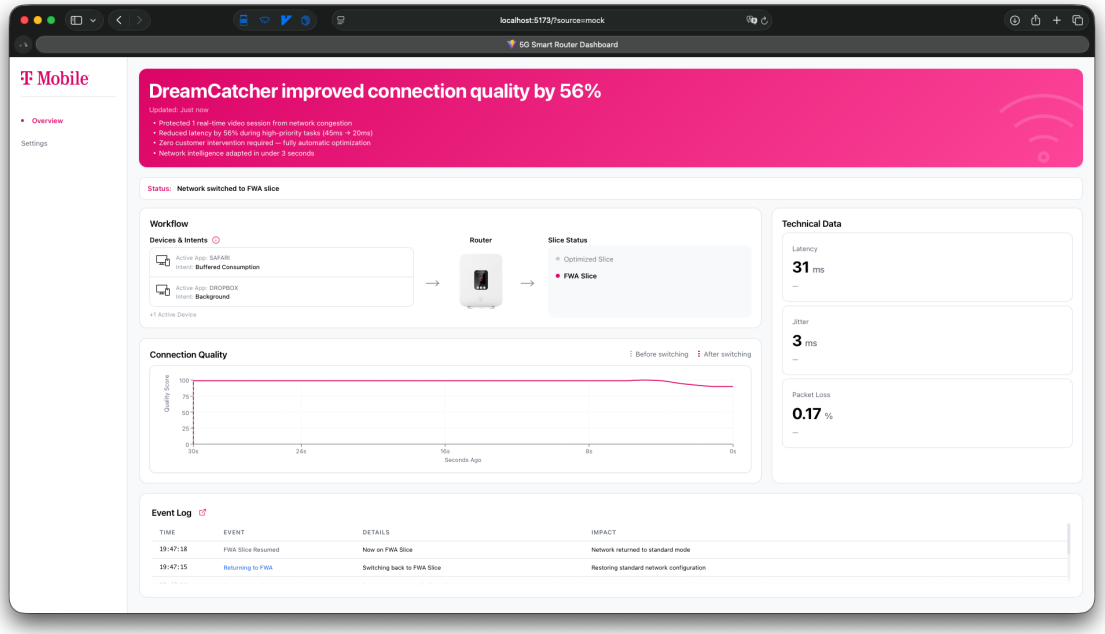
```
cd dream-catcher-tui
BACKEND_INGEST_URL=http://127.0.0.1:3002/api/ingest/event npm run dev -- --demo
```

Terminal 3 — Start the business dashboard:

```
cd dream-catcher-frontend-business
npm run dev
```

Open your browser at `http://localhost:5173`.

What you will see: The business dashboard loads and immediately begins receiving live telemetry driven by the TUI demo. As the cycle progresses, the quality-over-time chart updates in real time, switch markers annotate the chart at the exact moment the slice changes, and the metric cards refresh every second.



4.4 Mode C — Backend with Mock Data and Dashboard

No TUI needed. The backend runs its own built-in 41-second scripted scenario.

```
# Terminal 1
cd dream-catcher-backend && npm run dev

# Terminal 2
cd dream-catcher-frontend-business && npm run dev
```

Open `http://localhost:5173`. The dashboard behaves identically to Mode B.

4.5 Mode D — End-User Dashboard

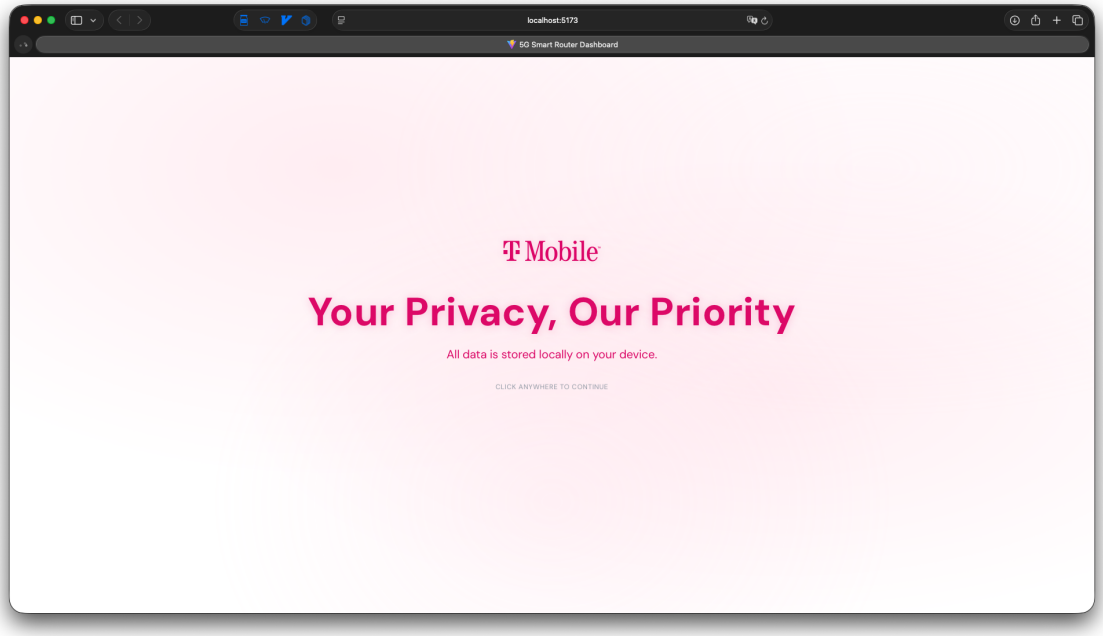
Step 1. Start both the local API and frontend together:

```
cd dream-catcher-frontend-user
npm run dev:all
```

This starts the local Express API on port 3001 and the Vite frontend on port 5173.

Step 2. Open `http://localhost:5173`.

What you will see: A consumer-facing dashboard displaying router status, the detected activity, the active network slice, weekly performance statistics, and a live optimization log. You can toggle between Automatic and Custom priority modes, and switch between Work and Games optimization goals.



4.6 Mode E — Live Mode (Real Traffic)

Warning: This mode requires a pfSense router, Python dependencies, and root privileges. Complete Sections 3.5 and 3.6 before proceeding.

Step 1. Identify your network interface name:

```
ip link show     # Linux
ifconfig         # macOS
```

Common interface names: `eth0`, `ens19`, `en0`

Step 2. Start the backend in TUI ingest mode:

```
cd dream-catcher-backend
TELEMETRY_SOURCE=tui npm run dev
```

Step 3. Start the TUI in live mode on your interface (root required):

```
cd dream-catcher-tui
BACKEND_INGEST_URL=http://127.0.0.1:3002/api/ingest/event sudo npm run dev -- -i
    ens19
```

Step 4. Start the business dashboard:

```
cd dream-catcher-frontend-business && npm run dev
# Open http://localhost:5173
```

The system is now monitoring real traffic. Start a Zoom or Teams call from any device on the network and watch the slice switch automatically in the terminal and dashboard.

4.7 Switching Dashboard Views

The business frontend codebase contains both the business and end-user dashboard layouts. To switch views, edit `src/App.jsx` in `dream-catcher-frontend-business`:

```
// Business stakeholder view (default)
const dashboardType = 'business'

// End-user router view
const dashboardType = 'user'
```

Save the file — Vite will hot-reload the browser automatically.

4.8 Shutting Down

- **TUI:** press **q** in the terminal window
- **Backend / Frontend servers:** press **Ctrl+C** in their respective terminal windows
- In full-stack mode, shut down the backend before the TUI to avoid connection error messages

5 Technical Specifications

5.1 System Architecture

```
+-------------------------------------------------------------+
|                          Clients                            |
|   Browser (Business Dashboard)  Browser (User Dashboard)    |
+-----------------------------+-------------------------------+
                              |  HTTP / WebSocket
                              v
+-------------------------------------+
|        dream-catcher-backend        |
|   Express REST API   (port 3002)    |
|   WebSocket Server  /ws/telemetry   |
+--------------+----------------------+
               |  POST /api/ingest/event   (TUI mode)
               v
+-------------------------------------+
|         dream-catcher-tui           |
|   Ink Terminal UI   (TypeScript)    |
|                                     |
|   useDemoEngine  --- demo mode      |
|   usePythonBridge -- live mode      |
+---------------------+---------------+
                      |  NDJSON (stdout)
                      v
+-------------------------------------+
|      bridge/ndjson_bridge.py        |
|   NFStream  deep packet inspection  |
|   Intent classification (nDPI)      |
|   WAN switching via pfSense API     |
+-------------------------------------+
```

5.2 Technologies Used

Layer	Technology	Version
Backend HTTP server	Express	4.x
Backend WebSocket	ws	8.x
Backend network metrics	ping (ICMP)	—
Frontend framework	React	18.x
Frontend build tool	Vite	4.x
Frontend charts	Recharts	3.x
Frontend animations	Framer Motion	12.x
Terminal UI	Ink	5.x
TUI language	TypeScript	5.x
TUI CLI parsing	meow	13.x
Python DPI	NFStream / nDPI	—
Python networking	scapy, psutil	—
Python pfSense client	requests	—

5.3 Data Flow

Mock / demo mode:

1. `telemetrySimulator.js` generates scripted telemetry on a fixed timeline
2. Backend REST routes and WebSocket serve this data to connected frontends
3. Frontend `useTelemetryStream` hook maintains a rolling time series and derives a quality score

TUI ingest mode:

1. TUI (demo or live) generates `BridgeEvent` objects
2. TUI POSTs each event to `POST /api/ingest/event`
3. Backend stores state in memory; REST routes and WebSocket serve the same state to all frontends
4. Frontends are data-source agnostic — they do not know whether data came from mock or TUI

5.4 WebSocket Message Format

Clients connect to `ws://localhost:3002/ws/telemetry`. The server pushes one `telemetry_tick` message per second:

```
{
  "type": "telemetry_tick",
  "data": {
    "state": {
      "currentSlice": "video",
      "decisionStatus": "stabilized"
    },
```

```
      "metrics": {
        "latencyMs": 20,
        "jitterMs": 2,
        "throughputMbps": 55,
        "packetLossPct": 0.05,
        "qualityScore": 95
      },
      "event": "MEETING_DETECTED"
    },
    "timestamp": "2024-01-22T10:30:00.000Z"
  }
```


5.5 Intent Classification Categories

The TUI classifies each detected network flow into one of five intent categories. WAN switching is triggered exclusively by **Conversational Real-time** traffic.

Category	Colour	Examples
Conversational Real-time	Red	Zoom, FaceTime, Microsoft Teams
Interactive Workflows	Yellow	Slack, GitHub
Interactive Entertainment	Cyan	Steam, Twitch
Buffered Consumption	Blue	YouTube, Netflix, Spotify
Background	Gray	DNS, NTP, iCloud sync

5.6 Demo Scenario Timeline

Both the backend built-in simulator and the TUI demo engine run the same scripted cycle:

Time	Phase	What Happens
0–10s	Idle	Mixed background traffic, WAN1 active
10s	Detect	Real-time traffic flow appears (Zoom)
10–13s	Switching	WAN priority switches to WAN2
13s	Stabilized	Low-latency slice active, quality improves
13–30s	Active	Optimised operation, metrics held stable
30–40s	Cooldown	Real-time traffic ends, cooldown timer runs
40s	Revert	WAN priority reverts to WAN1
40–45s	Idle	Brief idle before cycle repeats

6 Security and Privacy

6.1 API Key Security

The pfSense API key grants full administrative control of your router. Treat it as you would any privileged password.

- **Do not commit it to source control.** The placeholder `YOUR_PFSENSE_API_KEY` in `tools/wan_toggle-v2.py` must be replaced locally and must never be pushed to any shared repository.
- For any deployment beyond a single local machine, store credentials in environment variables or a dedicated secrets manager.
- Rotate the key immediately if it is ever accidentally exposed.

6.2 Network Access

- The backend server binds to `127.0.0.1` by default and is not reachable from other machines on the network unless you explicitly change the `HOST` environment variable.
- The Vite development server also binds to localhost by default.

Important: Do not expose any Dream Catcher service to the public internet without first adding an authentication layer. These services are designed for use on a trusted local network only.

6.3 TLS and the pfSense Certificate

The Python bridge connects to pfSense over HTTPS. pfSense ships with a self-signed certificate by default, so the bridge disables TLS certificate verification (`verify=False`). This is acceptable on a private local network but leaves the connection vulnerable to machine-in-the-middle attacks on untrusted networks. For production deployments, install a valid certificate on pfSense and remove the `verify=False` flag from the bridge.

6.4 Packet Capture

Live mode captures all packets on the selected network interface, including unencrypted traffic from every device on the network. Only operate Dream Catcher in live mode on networks you own or have explicit written authorisation to monitor.

6.5 Data Privacy

- Dream Catcher does not transmit any captured traffic data outside your local machine.
- No telemetry, usage analytics, or diagnostic data are sent to any external service.
- All runtime data is held in memory only; nothing is written to disk and nothing persists across restarts.

6.6 Known Operational Security Limitations

- The backend REST API and WebSocket server have no authentication layer. Any client that can reach port 3002 can read telemetry data and post ingest events.
- The `PUT /api/router/config` endpoint accepts configuration changes but does not validate or apply them to a real router in mock mode.

7 Troubleshooting

7.1 The Backend Fails to Start

Symptom: `Error: listen EADDRINUSE :3002`

Cause: Another process is already using port 3002.

Fix: Find and terminate the conflicting process, or start the backend on a different port:

```
lsof -i :3002          # identify the process (macOS / Linux)
kill -9 <PID>          # stop it
# -- or --
PORT=3003 npm run dev
```

7.2 The Frontend Shows No Data

Symptom: Dashboard loads but metrics never appear or stay at zero.

Cause: The frontend cannot reach the backend.

Fix:

1. Confirm the backend is running: `curl http://localhost:3002/api/health`
2. Open browser developer tools and check the Console for WebSocket errors.
3. Ensure no local firewall is blocking port 3002.

7.3 The TUI Shows a Blank Screen or Crashes

Symptom: Terminal goes blank then exits, possibly with a React or Ink error.

Cause: Terminal window is too small, or a dependency is missing.

Fix:

1. Resize the terminal window (minimum ~100 columns × 30 rows recommended).
2. Re-run `npm install` to ensure all dependencies are present.
3. Verify Node.js version: `node --version` must be 18 or later.

7.4 Live Mode Fails with "Permission Denied"

Symptom: `Operation not permitted` when starting live mode.

Cause: Raw packet capture requires root privileges.

Fix:

```
sudo npm run dev -- -i en0
```

7.5 pfSense API Returns 401 Unauthorized

Symptom: WAN switching fails; bridge logs show a 401 HTTP response.

Cause: The API key is incorrect or not set.

Fix:

1. Verify the key in `wan_toggle-v2.py` matches the value shown in pfSense under **System → API → Keys**.
2. Confirm the pfSense-pkg-API package is installed and the API is enabled.
3. Test the key directly from the command line:

```
curl -k -H "x-api-key: YOUR_KEY" https://<pfsense-ip>/api/v2/status/system
```

7.6 pfSense API Returns 403 Forbidden

Symptom: API key is accepted but all routing requests are rejected.

Cause: The API key does not have write permission on routing resources.

Fix: In pfSense under **System → API → Keys**, edit the key and grant write access to routing resources.

7.7 Gateway Group Name Not Found

Symptom: Bridge logs show `Gateway group 'WAN_FAILOVER' not found` and list available group names.

Cause: The `GATEWAY_GROUP_NAME` constant in `wan_toggle-v2.py` does not match the name as configured in pfSense.

Fix: Log in to pfSense, navigate to **System → Routing → Gateway Groups**, and copy the exact name — including capitalisation — into `wan_toggle-v2.py`.

7.8 TUI Not Forwarding Events to the Backend

Symptom: TUI runs and shows the demo, but the business dashboard stays at baseline and never shows a switch event.

Cause: `BACKEND_INGEST_URL` environment variable is not set.

Fix:

```
BACKEND_INGEST_URL=http://127.0.0.1:3002/api/ingest/event npm run dev -- --demo
```

7.9 NFStream Fails to Install

Symptom: `pip install nfstream` fails with a C build or compiler error.

Cause: NFStream requires a C build environment and is only fully supported on Linux.

Fix (Ubuntu / Debian):

```
sudo apt install build-essential libpcap-dev python3-dev
pip install nfstream
```

> **Note:** On macOS, NFStream support is limited. Use demo mode on macOS instead of attempting live mode.

8 Limitations

8.1 Demo Mode vs. Live Mode

The demo scenario is scripted and fully deterministic. It does not reflect the variability of real-world network conditions. All metrics shown in demo mode are simulated values and are not measurements from actual network hardware.

8.2 macOS Support for Live Mode

NFStream (the deep packet inspection engine) has limited support on macOS due to OS-level restrictions on raw packet capture. Live mode is only fully supported on Linux. On macOS, use demo mode or run the TUI on a dedicated Linux host.

8.3 No Authentication Layer

The backend REST API and WebSocket server ship without any authentication mechanism. They are intended solely for use on a trusted local network. Any device that can reach port 3002 can read telemetry data or post ingest events.

8.4 Single Gateway Group

The WAN switching logic targets one gateway group (`WAN_FAILOVER` by default). Multi-group or multi-WAN topologies involving more than two WANs require modifications to `wan_toggle-v2.py` .

8.5 In-Memory State Only

The backend holds all runtime state in memory. Restarting the backend resets all telemetry history, event logs, and ingest state. There is no database or persistent storage layer.

8.6 Fixed Intent Mapping

Application classification is based on a static `intent_mapping.yaml` configuration file. New applications or IP ranges not already covered must be added manually to this file to be correctly classified.

8.7 Throughput Measurement Accuracy

Passive throughput measurement (the default mode) reads OS-level byte counters from the selected network interface. This reflects all traffic on that interface, not just flows from monitored applications, and may not accurately represent the throughput of individual applications.

8.8 Browser Compatibility

The frontend dashboards are developed and tested primarily on Chrome. Other modern browsers (Firefox, Safari, Edge) are generally supported but have not been systematically tested against all dashboard features.

8.9 pfSense Version Compatibility

The pfSense REST API integration targets the pfSense-pkg-API v2 endpoint schema. This is a third-party package and may not be compatible with all pfSense versions or future releases. The integration has been tested against pfSense 2.7.